

# 第9章: スタイリング & UI ポリッシュ

機能が完成したアプリの見た目と使い心地を本格的に仕上げる章です。「動くだけ」のアプリを「使いたくなる」アプリに変えていきます。

## この章で学ぶこと

| テーマ             | 内容                                                           | なぜ重要か                 |
|-----------------|--------------------------------------------------------------|-----------------------|
| Tailwind CSS 実践 | CSSカスタムプロパティ（CSS変数： <code>--変数名</code> で色やサイズを一元管理する仕組み）の設定  | デザインの一貫性を保つため         |
| レスポンシブデザイン      | 画面サイズに応じてレイアウトを変える仕組み。スマホ/タブレット/PC全てに対応                      | ユーザーは様々なデバイスでアクセスするため |
| UIコンポーネント改善     | トースト通知（操作結果を一時的に表示するメッセージ）、ページネーション（大量データを複数ページに分割表示）        | ユーザー体験の向上のため          |
| アニメーション         | ホバー（マウスを乗せた時）エフェクト、トランジション（CSS Transition：プロパティの変化を滑らかにする機能） | 操作のフィードバックを伝えるため      |
| ダークモード          | 背景が暗い配色に切り替わるモード。目の負担を軽減                                     | ユーザーの好みに対応するため        |
| アクセシビリティ        | a11y（Accessibility：障害のある方を含む全ての人がWebを利用できるようにすること）対応         | すべての人に使いやすいアプリにするため   |

書籍管理アプリの見た目を本格的に仕上げていく章です。Tailwind CSS を活用したレスポンシブデザイン、アニメーション、ダークモード、アクセシビリティ（Accessibility：Webコンテンツをすべての人が利用できるようにするための取り組み）まで網羅的に学びます。

**デザインが重要な理由：** 同じ機能のアプリでも、見た目が整っているだけで「使いやすい」「信頼できそう」という印象を与えます。ポートフォリオ（作品集）に載せる際も、デザインの良さは大きなアピールポイントになります。

## 目次

- 0. 前提知識: CSSとTailwindの超基礎
- 1. Tailwind CSS 実践
- 2. レスポンシブデザイン

3. UI コンポーネントの改善
  4. アニメーション
  5. ダークモード（発展）
  6. アクセシビリティ
  7. 最終的な画面の説明
  8. コンポーネント構成図（最終版）
- 

## 0. 前提知識: CSSとTailwindの超基礎

---

### 0.1 CSSって何？

CSS（Cascading Style Sheets）はHTMLの「見た目」を指定する言語です。色・フォント・余白・配置などを記述します。

```
<!-- HTML -->
<button class="btn-primary">送信</button>
```

```
/* CSS */
.btn-primary {
  background-color: #3b82f6; /* 背景色: 青 */
  color: white;               /* 文字色: 白 */
  padding: 8px 16px;         /* 内側の余白 */
  border-radius: 8px;        /* 角丸 */
  border: none;              /* 枠線なし */
}
```

▼ 表示結果:



← 青背景、白文字、角丸のボタン

## 0.2 主要なCSSプロパティ

| プロパティ                         | 何を変える？ | 例                                    |
|-------------------------------|--------|--------------------------------------|
| <code>color</code>            | 文字色    | <code>color: red;</code>             |
| <code>background-color</code> | 背景色    | <code>background-color: #fff;</code> |
| <code>font-size</code>        | 文字サイズ  | <code>font-size: 16px;</code>        |
| <code>font-weight</code>      | 文字の太さ  | <code>font-weight: bold;</code>      |
| <code>padding</code>          | 内側の余白  | <code>padding: 8px 16px;</code>      |
| <code>margin</code>           | 外側の余白  | <code>margin: 16px;</code>           |
| <code>border</code>           | 枠線     | <code>border: 1px solid #ccc;</code> |
| <code>border-radius</code>    | 角丸     | <code>border-radius: 8px;</code>     |
| <code>display</code>          | 表示方式   | <code>display: flex;</code>          |
| <code>width / height</code>   | 幅・高さ   | <code>width: 100%;</code>            |

## 0.3 Tailwind CSS とは

通常のCSSは「CSSファイルを別に作って、クラスを定義して、HTMLに `class` を書く」という流れですが、これを毎回やるとクラス命名で消耗します。

Tailwind CSS は「あらかじめ大量のクラスが用意されていて、`bg-blue-500` のようなクラス名をHTMLに直接書くだけでスタイルが当たる」という仕組みです（ユーティリティファーストと言います）。

▼ 同じボタンを Tailwind で書くと:

```
<button class="bg-blue-500 text-white px-4 py-2 rounded-lg">
  送信
</button>
```

CSSファイルを書く必要がありません。

## 0.4 Tailwindクラス命名のパターン

ほぼ全クラスは「プロパティの省略 + `-` + 値」の形をしています。

| クラス                            | 意味               | 通常のCSS換算                                                  |
|--------------------------------|------------------|-----------------------------------------------------------|
| <code>bg-red-500</code>        | 背景色を赤500         | <code>background-color: #ef4444;</code>                   |
| <code>text-white</code>        | 文字色を白            | <code>color: white;</code>                                |
| <code>text-xl</code>           | 文字サイズXL          | <code>font-size: 1.25rem;</code>                          |
| <code>p-4</code>               | 全方向 padding 16px | <code>padding: 1rem;</code>                               |
| <code>px-4</code>              | 横方向 padding      | <code>padding-left: 1rem; padding-right: 1rem;</code>     |
| <code>py-2</code>              | 縦方向 padding      | <code>padding-top: 0.5rem; padding-bottom: 0.5rem;</code> |
| <code>m-4</code>               | 全方向 margin       | <code>margin: 1rem;</code>                                |
| <code>rounded-lg</code>        | 角丸 大             | <code>border-radius: 0.5rem;</code>                       |
| <code>flex</code>              | flexコンテナに        | <code>display: flex;</code>                               |
| <code>items-center</code>      | 縦方向中央揃え          | <code>align-items: center;</code>                         |
| <code>gap-2</code>             | 子要素間の隙間          | <code>gap: 0.5rem;</code>                                 |
| <code>w-full</code>            | 幅100%            | <code>width: 100%;</code>                                 |
| <code>hover:bg-blue-700</code> | マウスホバー時の色        | <code>:hover { background: ...; }</code>                  |

## 0.5 数値スケール

Tailwind の数値（`p-4` , `text-xl`）は4の倍数のpxなどの規則があります。

| Tailwind               | 値        | px換算 |
|------------------------|----------|------|
| <code>p-1</code>       | 0.25rem  | 4px  |
| <code>p-2</code>       | 0.5rem   | 8px  |
| <code>p-4</code>       | 1rem     | 16px |
| <code>p-8</code>       | 2rem     | 32px |
| <code>text-xs</code>   | 0.75rem  | 12px |
| <code>text-sm</code>   | 0.875rem | 14px |
| <code>text-base</code> | 1rem     | 16px |
| <code>text-lg</code>   | 1.125rem | 18px |
| <code>text-xl</code>   | 1.25rem  | 20px |
| <code>text-2xl</code>  | 1.5rem   | 24px |

## 0.6 色のスケール

色は `red-50` ～ `red-950` の11段階で、数字が大きいほど濃くなります。

50 100 200 300 400 500 600 700 800 900 950  
 ↑ 一番薄い                      ↑ 標準                      ↑ 一番濃い

コツ: 「背景は薄い色 (100, 200)、文字は濃い色 (700, 800, 900)」という組み合わせがおすすめ。 `bg-blue-100 text-blue-800` で柔らかい青のバッジになります。

## 0.7 レスポンシブの書き方

「PCでは横並び、スマホでは縦並び」のように画面サイズで変えるには **接頭辞** を付けます。

```
<div class="flex-col md:flex-row">
  <!-- スマホ: 縦並び (flex-col) -->
  <!-- 768px 以上 (md): 横並び (flex-row) -->
</div>
```

| 接頭辞              | 何以上で適用？   | 想定デバイス |
|------------------|-----------|--------|
| <code>sm:</code> | 640px 以上  | 大きめスマホ |
| <code>md:</code> | 768px 以上  | タブレット  |
| <code>lg:</code> | 1024px 以上 | ノートPC  |
| <code>xl:</code> | 1280px 以上 | デスクトップ |

## 1. Tailwind CSS 実践

### 1.1 globals.css のカスタマイズ

`app/globals.css` にアプリ全体で使うカスタムスタイルを定義します。CSS カスタムプロパティ（変数）を使い、ライトモードとダークモードの両方に対応できる設計にします。

```

/* app/globals.css */

@tailwind base;
@tailwind components;
@tailwind utilities;

/* =====
1. CSS カスタムプロパティ (カラートークン)
===== */
:root {
  /* プライマリカラー (インディゴ系) */
  --color-primary-50: #eef2ff;
  --color-primary-100: #e0e7ff;
  --color-primary-200: #c7d2fe;
  --color-primary-300: #a5b4fc;
  --color-primary-400: #818cf8;
  --color-primary-500: #6366f1;
  --color-primary-600: #4f46e5;
  --color-primary-700: #4338ca;
  --color-primary-800: #3730a3;
  --color-primary-900: #312e81;
  --color-primary-950: #1e1b4b;

  /* セカンダリカラー (エメラルド系) */
  --color-secondary-50: #ecfdf5;
  --color-secondary-100: #d1fae5;
  --color-secondary-200: #a7f3d0;
  --color-secondary-300: #6ee7b7;
  --color-secondary-400: #34d399;
  --color-secondary-500: #10b981;
  --color-secondary-600: #059669;
  --color-secondary-700: #047857;
  --color-secondary-800: #065f46;
  --color-secondary-900: #064e3b;
  --color-secondary-950: #022c22;

  /* 背景・テキスト */
  --color-background: #ffffff;
  --color-foreground: #0f172a;
  --color-muted: #64748b;
  --color-muted-foreground: #94a3b8;
  --color-border: #e2e8f0;
  --color-card: #ffffff;
  --color-card-foreground: #0f172a;

  /* 状態カラー */
  --color-success: #10b981;
  --color-warning: #f59e0b;
  --color-error: #ef4444;

```

```

--color-info: #3b82f6;

/* シャドウ */
--shadow-sm: 0 1px 2px 0 rgb(0 0 0 / 0.05);
--shadow-md: 0 4px 6px -1px rgb(0 0 0 / 0.1), 0 2px 4px -2px rgb(0 0 0 / 0.1);
--shadow-lg: 0 10px 15px -3px rgb(0 0 0 / 0.1), 0 4px 6px -4px rgb(0 0 0 / 0.1);

/* ボーダー半径 */
--radius-sm: 0.375rem;
--radius-md: 0.5rem;
--radius-lg: 0.75rem;
--radius-xl: 1rem;
}

/* ダークモード用のカスタムプロパティ */
.dark {
  --color-background: #0f172a;
  --color-foreground: #f8fafc;
  --color-muted: #94a3b8;
  --color-muted-foreground: #64748b;
  --color-border: #334155;
  --color-card: #1e293b;
  --color-card-foreground: #f8fafc;

  --shadow-sm: 0 1px 2px 0 rgb(0 0 0 / 0.3);
  --shadow-md: 0 4px 6px -1px rgb(0 0 0 / 0.4), 0 2px 4px -2px rgb(0 0 0 / 0.3);
  --shadow-lg: 0 10px 15px -3px rgb(0 0 0 / 0.4), 0 4px 6px -4px rgb(0 0 0 / 0.3);
}

/* =====
2. ベースレイヤー
===== */
@layer base {
  /* HTML・Body のリセットと基本設定 */
  html {
    scroll-behavior: smooth;
    -webkit-font-smoothing: antialiased;
    -moz-osx-font-smoothing: grayscale;
  }

  body {
    @apply bg-[var(--color-background)] text-[var(--color-foreground)];
    font-feature-settings: "rlig" 1, "calt" 1;
    transition: background-color 0.3s ease, color 0.3s ease;
  }

  /* フォーカスリングのデフォルトスタイル */
  *:focus-visible {
    @apply outline-2 outline-offset-2 outline-primary-500;
  }
}

```



```

/* 見出しのデフォルトスタイル */
h1 {
  @apply text-3xl font-bold tracking-tight;
}

h2 {
  @apply text-2xl font-semibold tracking-tight;
}

h3 {
  @apply text-xl font-semibold;
}

/* リンクのデフォルトスタイル */
a {
  @apply text-primary-600 hover:text-primary-700 dark:text-primary-400 dark:hover:text-p
  transition: color 0.15s ease;
}
}

/* =====
3. コンポーネントレイヤー
===== */
@layer components {
  /* ボタンの共通スタイル */
  .btn {
    @apply inline-flex items-center justify-center
      rounded-lg px-4 py-2
      text-sm font-medium
      transition-all duration-200 ease-in-out
      focus-visible:outline-none focus-visible:ring-2
      focus-visible:ring-offset-2
      disabled:pointer-events-none disabled:opacity-50;
  }

  .btn-primary {
    @apply btn
      bg-primary-600 text-white
      hover:bg-primary-700
      active:bg-primary-800
      focus-visible:ring-primary-500;
  }

  .btn-secondary {
    @apply btn
      bg-secondary-600 text-white
      hover:bg-secondary-700
      active:bg-secondary-800
      focus-visible:ring-secondary-500;
  }
}

```

```

}

.btn-outline {
  @apply btn
    border-2 border-primary-600 text-primary-600
    hover:bg-primary-50
    active:bg-primary-100
    dark:purple-border-primary-400 dark:text-primary-400
    dark:hover:bg-primary-950
    focus-visible:ring-primary-500;
}

.btn-danger {
  @apply btn
    bg-red-600 text-white
    hover:bg-red-700
    active:bg-red-800
    focus-visible:ring-red-500;
}

.btn-ghost {
  @apply btn
    text-gray-600 hover:bg-gray-100
    dark:text-gray-400 dark:hover:bg-gray-800
    focus-visible:ring-gray-500;
}

/* カードの共通スタイル */
.card {
  @apply rounded-xl border border-[var(--color-border)]
    bg-[var(--color-card)] text-[var(--color-card-foreground)]
    shadow-sm;
  transition: box-shadow 0.2s ease, transform 0.2s ease;
}

.card-hover {
  @apply card hover:shadow-lg hover:-translate-y-1;
}

/* 入力フィールドの共通スタイル */
.input {
  @apply w-full rounded-lg border border-[var(--color-border)]
    bg-[var(--color-background)]
    px-4 py-2.5
    text-sm
    placeholder:text-[var(--color-muted-foreground)]
    focus:purple-border-primary-500 focus:outline-none focus:ring-2
    focus:ring-primary-500/20
    disabled:cursor-not-allowed disabled:opacity-50
    dark:focus:purple-border-primary-400 dark:focus:ring-primary-400/20;
}

```

```

    transition: border-color 0.15s ease, box-shadow 0.15s ease;
  }

/* パツジ */
.badge {
  @apply inline-flex items-center rounded-full
    px-2.5 py-0.5
    text-xs font-medium;
}

.badge-primary {
  @apply badge bg-primary-100 text-primary-700
    dark:bg-primary-900 dark:text-primary-300;
}

.badge-success {
  @apply badge bg-green-100 text-green-700
    dark:bg-green-900 dark:text-green-300;
}

.badge-warning {
  @apply badge bg-yellow-100 text-yellow-700
    dark:bg-yellow-900 dark:text-yellow-300;
}

.badge-error {
  @apply badge bg-red-100 text-red-700
    dark:bg-red-900 dark:text-red-300;
}
}

/* =====
4. ユーティリティレイヤー
===== */
@layer utilities {
/* テキスト省略（複数行対応） */
.line-clamp-1 {
  display: -webkit-box;
  -webkit-line-clamp: 1;
  -webkit-box-orient: vertical;
  overflow: hidden;
}

.line-clamp-2 {
  display: -webkit-box;
  -webkit-line-clamp: 2;
  -webkit-box-orient: vertical;
  overflow: hidden;
}
}

```

```

.line-clamp-3 {
  display: -webkit-box;
  -webkit-line-clamp: 3;
  -webkit-box-orient: vertical;
  overflow: hidden;
}

/* スクロールバーのカスタマイズ */
.scrollbar-thin {
  scrollbar-width: thin;
  scrollbar-color: var(--color-muted-foreground) transparent;
}

.scrollbar-thin::-webkit-scrollbar {
  width: 6px;
  height: 6px;
}

.scrollbar-thin::-webkit-scrollbar-track {
  background: transparent;
}

.scrollbar-thin::-webkit-scrollbar-thumb {
  background-color: var(--color-muted-foreground);
  border-radius: 3px;
}

/* グラスモーフィズム効果 */
.glass {
  @apply bg-white/80 backdrop-blur-md dark:bg-gray-900/80;
}
}

/* =====
5. アニメーション定義
===== */
@keyframes fadeIn {
  from {
    opacity: 0;
    transform: translateY(8px);
  }
  to {
    opacity: 1;
    transform: translateY(0);
  }
}

@keyframes fadeOut {
  from {
    opacity: 1;

```

```

        transform: translateY(0);
    }
    to {
        opacity: 0;
        transform: translateY(8px);
    }
}

@keyframes slideInRight {
    from {
        opacity: 0;
        transform: translateX(16px);
    }
    to {
        opacity: 1;
        transform: translateX(0);
    }
}

@keyframes slideInUp {
    from {
        opacity: 0;
        transform: translateY(16px);
    }
    to {
        opacity: 1;
        transform: translateY(0);
    }
}

@keyframes scaleIn {
    from {
        opacity: 0;
        transform: scale(0.95);
    }
    to {
        opacity: 1;
        transform: scale(1);
    }
}

@keyframes spin {
    to {
        transform: rotate(360deg);
    }
}

@keyframes shimmer {
    0% {
        background-position: -200% 0;
    }
}

```

```
}
100% {
    background-position: 200% 0;
}
}

@keyframes toastSlideIn {
    from {
        opacity: 0;
        transform: translateX(100%);
    }
    to {
        opacity: 1;
        transform: translateX(0);
    }
}

@keyframes toastSlideOut {
    from {
        opacity: 1;
        transform: translateX(0);
    }
    to {
        opacity: 0;
        transform: translateX(100%);
    }
}
```

## 1.2 カスタムカラーパレットの設定

アプリでは以下のカラーパレットを採用しています。

| 用途       | カラー名       | ベースカラー          | 説明               |
|----------|------------|-----------------|------------------|
| メインアクション | primary    | インディゴ (#6366f1) | ボタン、リンク、フォーカスリング |
| 補助アクション  | secondary  | エメラルド (#10b981) | 成功状態、補助ボタン       |
| 成功       | success    | グリーン (#10b981)  | 操作成功の通知          |
| 警告       | warning    | アンバー (#f59e0b)  | 注意喚起             |
| エラー      | error      | レッド (#ef4444)   | エラーメッセージ、削除ボタン   |
| 情報       | info       | ブルー (#3b82f6)   | 情報表示             |
| テキスト     | foreground | スレート (#0f172a)  | メインテキスト          |
| 背景       | background | ホワイト (#ffffff)  | ページ背景            |
| ミュート     | muted      | スレート (#64748b)  | 補足テキスト           |

カラーパレットの設計指針:

- **一貫性:** 同じ意味を持つ色は全画面で統一する
- **コントラスト比:** WCAG 2.1 AA 基準（4.5:1 以上）を満たす
- **ダークモード対応:** 各カラーに明暗の両バリエーションを用意する

### 1.3 tailwind.config.ts のカスタマイズ

```
// tailwind.config.ts

import type { Config } from "tailwindcss";

const config: Config = {
  // ダークモードの切り替え方法
  // "class" を指定すると、<html> タグに "dark" クラスを付けることで切り替え可能
  darkMode: "class",

  // Tailwind が適用されるファイルのパス
  content: [
    "./pages/**/*.{js,ts,jsx,tsx,mdx}",
    "./components/**/*.{js,ts,jsx,tsx,mdx}",
    "./app/**/*.{js,ts,jsx,tsx,mdx}",
  ],

  theme: {
    // =====
    // コンテナの設定
    // =====
    container: {
      center: true, // コンテナを中央寄せ
      padding: {
        DEFAULT: "1rem",
        sm: "2rem",
        lg: "4rem",
        xl: "5rem",
        "2xl": "6rem",
      },
    },
    screens: {
      sm: "640px",
      md: "768px",
      lg: "1024px",
      xl: "1280px",
      "2xl": "1400px", // デフォルトの1536pxより少し狭く
    },
  },

  extend: {
    // =====
    // カスタムカラーパレット
    // =====
    colors: {
      // プライマリカラー（インディゴ系）
      primary: {
        50: "var(--color-primary-50)",
        100: "var(--color-primary-100)",
        200: "var(--color-primary-200)",
      },
    },
  },
}
```



```

    300: "var(--color-primary-300)",
    400: "var(--color-primary-400)",
    500: "var(--color-primary-500)",
    600: "var(--color-primary-600)",
    700: "var(--color-primary-700)",
    800: "var(--color-primary-800)",
    900: "var(--color-primary-900)",
    950: "var(--color-primary-950)",
  },
  // セカンダリカラー (エメラルド系)
  secondary: {
    50: "var(--color-secondary-50)",
    100: "var(--color-secondary-100)",
    200: "var(--color-secondary-200)",
    300: "var(--color-secondary-300)",
    400: "var(--color-secondary-400)",
    500: "var(--color-secondary-500)",
    600: "var(--color-secondary-600)",
    700: "var(--color-secondary-700)",
    800: "var(--color-secondary-800)",
    900: "var(--color-secondary-900)",
    950: "var(--color-secondary-950)",
  },
  // セマンティックカラー
  background: "var(--color-background)",
  foreground: "var(--color-foreground)",
  muted: {
    DEFAULT: "var(--color-muted)",
    foreground: "var(--color-muted-foreground)",
  },
  border: "var(--color-border)",
  card: {
    DEFAULT: "var(--color-card)",
    foreground: "var(--color-card-foreground)",
  },
  success: "var(--color-success)",
  warning: "var(--color-warning)",
  error: "var(--color-error)",
  info: "var(--color-info)",
},

// =====
// カスタムフォント
// =====
fontFamily: {
  sans: [
    "Inter",
    "Noto Sans JP",
    "ui-sans-serif",
    "system-ui",
  ],

```

```

    "-apple-system",
    "sans-serif",
  ],
  mono: [
    "JetBrains Mono",
    "Fira Code",
    "ui-monospace",
    "monospace",
  ],
},

// =====
// カスタムフォントサイズ
// =====
fontSize: {
  "2xs": ["0.625rem", { lineHeight: "0.875rem" }],
},

// =====
// カスタムシャドウ
// =====
boxShadow: {
  sm: "var(--shadow-sm)",
  md: "var(--shadow-md)",
  lg: "var(--shadow-lg)",
  card: "0 2px 8px -2px rgb(0 0 0 / 0.08), 0 4px 12px -4px rgb(0 0 0 / 0.04)",
  "card-hover":
    "0 8px 24px -4px rgb(0 0 0 / 0.12), 0 4px 8px -4px rgb(0 0 0 / 0.08)",
},

// =====
// カスタムボーダー半径
// =====
borderRadius: {
  sm: "var(--radius-sm)",
  md: "var(--radius-md)",
  lg: "var(--radius-lg)",
  xl: "var(--radius-xl)",
},

// =====
// カスタムアニメーション
// =====
keyframes: {
  "fade-in": {
    from: { opacity: "0", transform: "translateY(8px)" },
    to: { opacity: "1", transform: "translateY(0)" },
  },
  "fade-out": {
    from: { opacity: "1", transform: "translateY(0)" },

```

```

    to: { opacity: "0", transform: "translateY(8px)" },
  },
  "slide-in-right": {
    from: { opacity: "0", transform: "translateX(16px)" },
    to: { opacity: "1", transform: "translateX(0)" },
  },
  "slide-in-up": {
    from: { opacity: "0", transform: "translateY(16px)" },
    to: { opacity: "1", transform: "translateY(0)" },
  },
  "scale-in": {
    from: { opacity: "0", transform: "scale(0.95)" },
    to: { opacity: "1", transform: "scale(1)" },
  },
  shimmer: {
    "0%": { backgroundPosition: "-200% 0" },
    "100%": { backgroundPosition: "200% 0" },
  },
  "toast-slide-in": {
    from: { opacity: "0", transform: "translateX(100%)" },
    to: { opacity: "1", transform: "translateX(0)" },
  },
  "toast-slide-out": {
    from: { opacity: "1", transform: "translateX(0)" },
    to: { opacity: "0", transform: "translateX(100%)" },
  },
},
animation: {
  "fade-in": "fade-in 0.3s ease-out",
  "fade-out": "fade-out 0.3s ease-out",
  "slide-in-right": "slide-in-right 0.3s ease-out",
  "slide-in-up": "slide-in-up 0.4s ease-out",
  "scale-in": "scale-in 0.2s ease-out",
  shimmer: "shimmer 2s infinite linear",
  "toast-in": "toast-slide-in 0.3s ease-out",
  "toast-out": "toast-slide-out 0.3s ease-in forwards",
  spin: "spin 1s linear infinite",
},

// =====
// カスタムスペーシング
// =====
spacing: {
  "18": "4.5rem",
  "88": "22rem",
  "128": "32rem",
},

// =====
// カスタムトランジション

```

```
// =====
transitionDuration: {
  "250": "250ms",
  "350": "350ms",
},
},
},
},

plugins: [],
};

export default config;
```

設定のポイント解説:

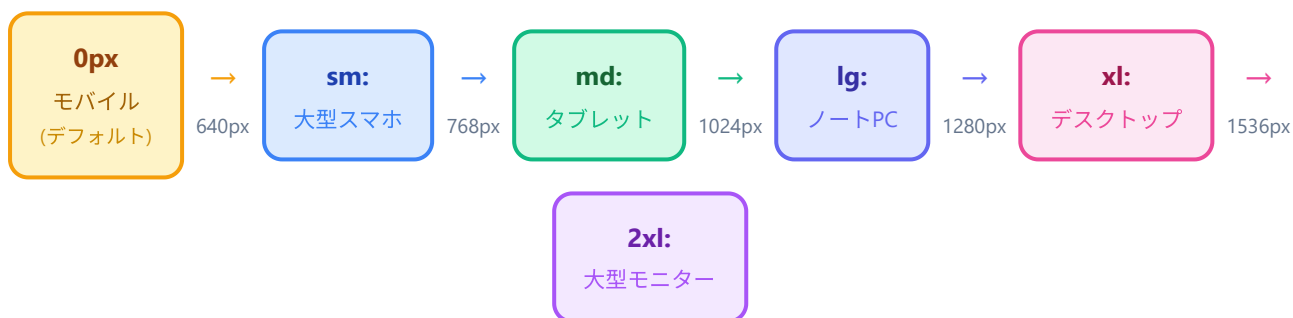
- `darkMode: "class"` -- `<html class="dark">` でダークモードを切り替える方式。 `media` にするとシステム設定に連動する
- `container` -- レスポンシブなコンテナ幅を画面サイズごとにカスタマイズ
- `colors` -- CSS カスタムプロパティ経由でカラーを定義することで、ダークモード切り替えが CSS 変数の上書きだけで済む
- `keyframes` / `animation` -- `globals.css` の `@keyframes` と合わせて、Tailwind のクラスとしてアニメーションを呼び出せるようにする

## 2. レスポンシブデザイン

### 2.1 Tailwind のブレイクポイント解説

Tailwind CSS はモバイルファースト設計です。何もプレフィックスを付けないスタイルがモバイル（最小画面）に適用され、`sm:` `md:` などのプレフィックスを付けるとそのブレイクポイント以上の画面幅で適用されます。

| プレフィックス | 最小幅    | CSS 相当                     | 想定デバイス         | 用途                  |
|---------|--------|----------------------------|----------------|---------------------|
| (なし)    | 0px    | @media (min-width: 0px)    | 小型スマートフォン      | デフォルト。全画面で適用        |
| sm:     | 640px  | @media (min-width: 640px)  | 大型スマートフォン（横持ち） | テキストサイズ微調整、パディング拡大  |
| md:     | 768px  | @media (min-width: 768px)  | タブレット          | 2カラムレイアウト、サイドバー表示   |
| lg:     | 1024px | @media (min-width: 1024px) | ノートPC          | 3カラムレイアウト、ナビゲーション展開 |
| xl:     | 1280px | @media (min-width: 1280px) | デスクトップ         | 4カラムレイアウト、広いマージン    |
| 2xl:    | 1536px | @media (min-width: 1536px) | 大型モニター         | 最大幅レイアウト            |



重要な考え方: モバイルファースト

```

/* 悪い例: デスクトップから書いて、小さい画面を上書き */
className="grid-cols-4 md:grid-cols-2 sm:grid-cols-1" ← 動かない

/* 良い例: モバイルから書いて、大きい画面を上書き */
className="grid-cols-1 md:grid-cols-2 lg:grid-cols-4" ← 正しい

```

## 2.2 書籍カードのグリッドをレスポンス対応

```
// =====  
// ファイルパス: src/components/BookGrid.tsx  
// 役割      : 書籍カードを「画面サイズに応じて列数が変わる格子状」に並べる  
// -----  
// Tailwind CSS のレスポンス接頭辞を使うことで、メディアクエリを書かずに  
// 「PCでは4列、タブレットでは3列、スマホでは1列」のような切り替えができる。  
// =====  
  
"use client";  
  
import Link from "next/link";  
import { Book } from "@types/book";  
import { BookCard } from "../BookCard";  
import { EmptyState } from "../EmptyState";  
  
type BookGridProps = {  
  books: Book[]; // 表示する書籍配列  
};  
  
export function BookGrid({ books }: BookGridProps) {  
  // (1) 0冊なら専用の空状態コンポーネントを出して終わる（早期リターン）  
  if (books.length === 0) {  
    return <EmptyState />;  
  }  
  
  // (2) 1冊以上なら格子レイアウトで描画  
  return (  
    <div  
      // ▼ Tailwind クラスの読み書き ▼  
      // grid          : display: grid (CSSグリッドレイアウト)  
      // grid-cols-1    : 列数 = 1 (モバイル既定)  
      // gap-4          : マス目同士の間隔 16px  
      // sm:grid-cols-2 : 640px以上で 2 列に切替  
      // sm:gap-5       : 640px以上で 隙間 20px  
      // lg:grid-cols-3 : 1024px以上で 3 列  
      // lg:gap-6       : 1024px以上で 隙間 24px  
      // xl:grid-cols-4 : 1280px以上で 4 列  
      // 「sm:」 「lg:」などはメディアクエリを意味する接頭辞。  
      className=  
        grid  
        grid-cols-1  
        gap-4  
        sm:grid-cols-2  
        sm:gap-5  
        lg:grid-cols-3  
        lg:gap-6  
        xl:grid-cols-4  
      =
```

```

>
{ /*
  (3) 配列を map で1件ずつ <Link> + <BookCard> に展開する
    ・key={book.id}      : Reactのリスト識別キー (必須)
    ・href={...}         : クリックで詳細ページへ
    ・className="group"  : 子要素の "group-hover:..." を有効化する目印
    ・animationDelay     : i番目のカードを i*50ms 遅らせて順次フェードイン
*/}
{books.map((book, index) => (
  <Link
    key={book.id}
    href={`\books/${book.id}`}
    className="group block"
    style={{
      animationDelay: `${index * 50}ms`,
    }}
  >
    <BookCard book={book} />
  </Link>
  )
)}
</div>
);
}

```

```
// components/BookCard.tsx

"use client";

import { Book } from "@types/book";

type BookCardProps = {
  book: Book;
};

export function BookCard({ book }: BookCardProps) {
  // 読了状況に応じてバッジのスタイルを切り替え
  const statusBadge = () => {
    switch (book.status) {
      case "reading":
        return <span className="badge-primary">読書中</span>;
      case "completed":
        return <span className="badge-success">読了</span>;
      case "want_to_read":
        return <span className="badge-warning">読みたい</span>;
      default:
        return null;
    }
  };

  return (
    <article
      className="
        card-hover
        animate-fade-in
        flex flex-col
        overflow-hidden
        p-0
      "
    >
      { /* 書籍のサムネイルエリア */ }
      <div
        className="
          relative
          aspect-[3/4]
          w-full
          overflow-hidden
          bg-gradient-to-br from-primary-100 to-primary-200
          dark:from-primary-900 dark:to-primary-800
        "
      >
        { /* サムネイル画像がある場合 */ }
        {book.thumbnailUrl ? (
          <img

```



```

src={book.thumbnailUrl}
alt={`「${book.title}」の表紙`}
className="
  h-full w-full
  object-cover
  transition-transform duration-300
  group-hover:scale-105
"
loading="lazy"
/>
) : (
  /* プレースホルダー */
  <div
    className="
      flex h-full w-full
      flex-col items-center justify-center
      gap-2
      p-4
      text-primary-400
      dark:text-primary-600
    "
  >
    <svg
      className="h-12 w-12"
      fill="none"
      viewBox="0 0 24 24"
      stroke="currentColor"
      strokeWidth={1.5}
      aria-hidden="true"
    >
      <path
        strokeLinecap="round"
        strokeLinejoin="round"
        d="M12 6.042A8.967 8.967 0 06 3.75c-1.052 0-2.062.18-3 .512v14.25A8.987 8
      />
    </svg>
    <span className="text-xs font-medium">No Image</span>
  </div>
)}

{/* ステータスバッジ (右上に配置) */}
<div className="absolute right-2 top-2">
  {statusBadge()}
</div>
</div>

{/* 書籍情報エリア */}
<div className="flex flex-1 flex-col gap-1.5 p-4">
  /* タイトル */
  <h3

```



```
};  
}
```

## 2.3 モバイル/タブレット/デスクトップでの表示の違い

各画面サイズでのグリッドレイアウトの変化を以下にまとめます。

モバイル（～639px）：`grid-cols-1`



- カードが1列で縦に並ぶ
- タイトルは `text-sm`（小さめ）
- パディングは最小限
- タッチ操作しやすい大きなタップ領域

タブレット（640px～1023px）：`sm:grid-cols-2`



- 2列のグリッド
- タイトルは `sm:text-base`（通常サイズ）

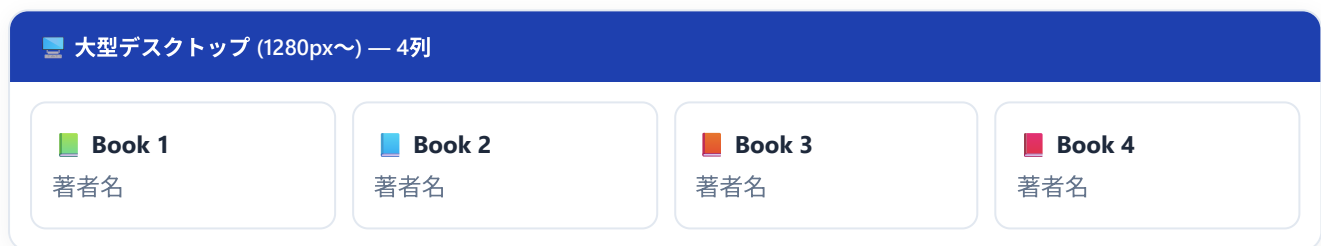
- ギャップが少し広がる (`sm:gap-5`)

デスクトップ (1024px~1279px) : `lg:grid-cols-3`



- 3列のグリッド
- ギャップがさらに広がる (`lg:gap-6`)

大型デスクトップ (1280px~) : `xl:grid-cols-4`



- 4列のグリッド
- 情報の一覧性が最大化

## 3. UI コンポーネントの改善

### 3.1 トースト通知の実装

操作結果（成功/エラー）をユーザーに伝えるトースト通知コンポーネントを作成します。React の Context API を使い、アプリのどこからでもトーストを呼び出せるようにします。

### 3.1.1 トーストの型定義

```
// types/toast.ts

export type ToastType = "success" | "error" | "warning" | "info";

export type Toast = {
  id: string;
  type: ToastType;
  title: string;
  message?: string;
  duration?: number; // ミリ秒。デフォルト 5000ms
};
```

### 3.1.2 トースト Context

```
// =====
// ファイルパス: src/contexts/ToastContext.tsx
// 役割      : アプリ全体で「トースト通知（一時的なメッセージ）」を扱う仕組み
// -----
// React の Context API を使うと、深い階層のコンポーネントにも
// プロパティをバケツリレーせずに値や関数を共有できる。
// ここでは「トーストを追加・削除する関数」を Context にして、
// どのコンポーネントからでも showToast(...) のように呼べるようにする。
// =====

// "use client" が必要な理由:
//   useState / useContext / setTimeout を使う＝ブラウザ側で動く必要がある。
"use client";

// React の Hook と型を取り込む
//   createContext: Context オブジェクトを作る関数
//   useContext    : Context の値を読む Hook
//   useState      : 状態管理 Hook
//   useCallback    : 関数を「依存変更時のみ再生成」するメモ化 Hook
//   ReactNode     : 「Reactで子要素として使えるあらゆる値」の型
import {
  createContext,
  useContext,
  useState,
  useCallback,
  type ReactNode,
} from "react";
import { Toast, ToastType } from "@/types/toast";

// -----
// (1) Context が提供する「価値」の型を定義
// -----
// この型に書かれた値・関数を、Context.Provider 経由で配布する。
type ToastContextType = {
  toasts: Toast[]; // 現在表示中のトースト
  addToast: ( // 任意のトーストを追加
    type: ToastType,
    title: string,
    message?: string,
    duration?: number
  ) => void;
  removeToast: (id: string) => void; // 指定IDを削除
  success: (title: string, message?: string) => void; // ↓ よく使う4種のショー
  error: (title: string, message?: string) => void;
  warning: (title: string, message?: string) => void;
  info: (title: string, message?: string) => void;
};
```

```

// -----
// (2) Context オブジェクトを作る
// -----
// 初期値は undefined (後で Provider が値を提供)。
// 取り出すときに undefined チェックを入れて安全にする (後述の useToast 参照)。
const ToastContext = createContext<ToastContextType | undefined>(undefined);

// -----
// (3) 一意なIDを作る簡易関数
// -----
// Date.now() (ミリ秒) と単調増加カウンタを組み合わせると衝突しないIDを作る。
// 大規模アプリでは uuid ライブラリを使うほうが安心。
let toastIdCounter = 0;
function generateToastId(): string {
  toastIdCounter += 1;
  return `toast-${Date.now()}-${toastIdCounter}`;
}

// -----
// (4) Provider コンポーネント
// -----
// アプリのルート付近で <ToastProvider> でラップすると、その内側の全ての
// コンポーネントが useToast() で値を受け取れる。
export function ToastProvider({ children }: { children: ReactNode }) {
  // (4-1) 表示中のトースト配列を state で保持
  const [toasts, setToasts] = useState<Toast[]>([]);

  // (4-2) 指定IDのトーストを削除する関数
  //       useCallback で関数の同一性を保つことで、依存配列に入れた箇所で
  //       不必要な再レンダリングが起きるのを防ぐ。
  const removeToast = useCallback((id: string) => {
    // 配列を「削除対象以外を残す」フィルタで更新
    setToasts((prev) => prev.filter((toast) => toast.id !== id));
  }, []);

  // (4-3) トーストを追加する関数
  //       duration 経過後に setTimeout で自動で削除する。
  const addToast = useCallback(
    (
      type: ToastType,
      title: string,
      message?: string,
      duration: number = 5000 // 既定で5秒で消える
    ) => {
      const id = generateToastId();
      const newToast: Toast = { id, type, title, message, duration };

      // 既存の配列に新トーストを末尾追加 (破壊しないよう新配列を作る)
      setToasts((prev) => [...prev, newToast]);
    }
  );
}

```

```

    // duration が 0 より大きいときだけ自動削除のタイマーをセット
    if (duration > 0) {
      setTimeout(() => {
        removeToast(id);
      }, duration);
    }
  },
  [removeToast] // removeToast が変わったら再生成
);

// ショートカットメソッド
const success = useCallback(
  (title: string, message?: string) => addToast("success", title, message),
  [addToast]
);

const error = useCallback(
  (title: string, message?: string) =>
    addToast("error", title, message, 8000), // エラーは長めに表示
  [addToast]
);

const warning = useCallback(
  (title: string, message?: string) => addToast("warning", title, message),
  [addToast]
);

const info = useCallback(
  (title: string, message?: string) => addToast("info", title, message),
  [addToast]
);

return (
  <ToastContext.Provider
    value={{ toasts, addToast, removeToast, success, error, warning, info }}
  >
    {children}
  </ToastContext.Provider>
);
}

export function useToast(): ToastContextType {
  const context = useContext(ToastContext);
  if (context === undefined) {
    throw new Error("useToast must be used within a ToastProvider");
  }
  return context;
}

```



### 3.1.3 トーストコンポーネント

```
// components/Toast.tsx

"use client";

import { useState, useEffect, useCallback } from "react";
import { Toast as ToastType } from "@types/toast";
import { useToast } from "@contexts/ToastContext";

// 各トーストタイプのアイコンとスタイル
const toastConfig = {
  success: {
    icon: (
      <svg
        className="h-5 w-5"
        fill="none"
        viewBox="0 0 24 24"
        stroke="currentColor"
        strokeWidth={2}
        aria-hidden="true"
      >
        <path
          strokeLinecap="round"
          strokeLinejoin="round"
          d="M9 12.75L11.25 15 15 9.75M21 12a9 9 0 11-18 0 9 9 0 0118 0z"
        />
      </svg>
    ),
    containerClass:
      "border-green-200 bg-green-50 dark:border-green-800 dark:bg-green-950",
    iconClass: "text-green-600 dark:text-green-400",
    titleClass: "text-green-800 dark:text-green-200",
    messageClass: "text-green-700 dark:text-green-300",
    progressClass: "bg-green-500",
  },
  error: {
    icon: (
      <svg
        className="h-5 w-5"
        fill="none"
        viewBox="0 0 24 24"
        stroke="currentColor"
        strokeWidth={2}
        aria-hidden="true"
      >
        <path
          strokeLinecap="round"
          strokeLinejoin="round"
          d="M12 9v3.75m9-.75a9 9 0 11-18 0 9 9 0 0118 0zm-9 3.75h.008v.008H12v-.008z"
        />
      </svg>
    ),
  },
};
```

```

    />
  </svg>
),
containerClass:
  "border-red-200 bg-red-50 dark:border-red-800 dark:bg-red-950",
iconClass: "text-red-600 dark:text-red-400",
titleClass: "text-red-800 dark:text-red-200",
messageClass: "text-red-700 dark:text-red-300",
progressClass: "bg-red-500",
},
warning: {
  icon: (
    <svg
      className="h-5 w-5"
      fill="none"
      viewBox="0 0 24 24"
      stroke="currentColor"
      strokeWidth={2}
      aria-hidden="true"
    >
      <path
        strokeLinecap="round"
        strokeLinejoin="round"
        d="M12 9v3.75m-9.303 3.376c-.866 1.5.217 3.374 1.948 3.374h14.71c1.73 0 2.813-1.
      />
    </svg>
  ),
  containerClass:
    "border-yellow-200 bg-yellow-50 dark:border-yellow-800 dark:bg-yellow-950",
  iconClass: "text-yellow-600 dark:text-yellow-400",
  titleClass: "text-yellow-800 dark:text-yellow-200",
  messageClass: "text-yellow-700 dark:text-yellow-300",
  progressClass: "bg-yellow-500",
},
info: {
  icon: (
    <svg
      className="h-5 w-5"
      fill="none"
      viewBox="0 0 24 24"
      stroke="currentColor"
      strokeWidth={2}
      aria-hidden="true"
    >
      <path
        strokeLinecap="round"
        strokeLinejoin="round"
        d="M11.25 11.25l.041-.02a.75.75 0 011.063.852l-.708 2.836a.75.75 0 001.063.853l
      />
    </svg>
  )

```

```

    },
    containerClass:
      "border-blue-200 bg-blue-50 dark:border-blue-800 dark:bg-blue-950",
    iconClass: "text-blue-600 dark:text-blue-400",
    titleClass: "text-blue-800 dark:text-blue-200",
    messageClass: "text-blue-700 dark:text-blue-300",
    progressClass: "bg-blue-500",
  },
};

```

// 個別のトーストアイテム

```

function ToastItem({ toast }: { toast: ToastType }) {
  const { removeToast } = useToast();
  const [isExiting, setIsExiting] = useState(false);
  const config = toastConfig[toast.type];

```

```

  const handleClose = useCallback(() => {
    setIsExiting(true);
    // アニメーション完了後に削除
    setTimeout(() => {
      removeToast(toast.id);
    }, 300);
  }, [removeToast, toast.id]);

```

// プログレスバー用のアニメーション

```

const duration = toast.duration || 5000;

```

```

return (
  <div
    role="alert"
    aria-live="assertive"
    aria-atomic="true"
    className={`
      relative
      flex w-full max-w-sm items-start gap-3
      overflow-hidden
      rounded-lg border
      p-4
      shadow-lg
      ${config.containerClass}
      ${isExiting ? "animate-toast-out" : "animate-toast-in"}
    `}
  >
    {/* アイコン */}
    <div className={`flex-shrink-0 ${config.iconClass}`}>
      {config.icon}
    </div>

    {/* テキスト内容 */}
    <div className="flex-1 min-w-0">

```

```

<p className={`text-sm font-semibold ${config.titleClass}`}>
  {toast.title}
</p>
{toast.message && (
  <p className={`mt-1 text-sm ${config.messageClass}`}>
    {toast.message}
  </p>
)}
</div>

{/* 閉じるボタン */}
<button
  type="button"
  onClick={handleClose}
  className={`
    flex-shrink-0
    rounded-md p-1
    opacity-70
    transition-opacity
    hover:opacity-100
    focus:outline-none focus:ring-2 focus:ring-offset-2
    ${config.iconClass}
  `}
  aria-label="通知を閉じる"
>
  <svg
    className="h-4 w-4"
    fill="none"
    viewBox="0 0 24 24"
    stroke="currentColor"
    strokeWidth={2}
    aria-hidden="true"
  >
    <path
      strokeLinecap="round"
      strokeLinejoin="round"
      d="M6 18L18 6M6 6L12 12"
    />
  </svg>
</button>

{/* プログレスバー（残り時間表示） */}
<div className="absolute bottom-0 left-0 right-0 h-1">
  <div
    className={`h-full ${config.progressClass} opacity-30`}
    style={{
      animation: `shrinkWidth ${duration}ms linear forwards`,
    }}
  />
</div>

```

```

    </div>
  );
}

// トーストコンテナ (画面右上に固定配置)
export function ToastContainer() {
  const { toasts } = useToast();

  if (toasts.length === 0) return null;

  return (
    <div
      aria-label="通知"
      className="
        fixed
        right-4
        top-4
        z-50
        flex
        flex-col
        gap-3
        sm:right-6
        sm:top-6
      "
    >
      {toasts.map((toast) => (
        <ToastItem key={toast.id} toast={toast} />
      ))}
    </div>
  );
}

```

使用例:

```
// 任意のコンポーネントから呼び出す
"use client";

import { useToast } from "@/contexts/ToastContext";

export function BookDeleteButton({ bookId }: { bookId: string }) {
  const toast = useToast();

  const handleDelete = async () => {
    try {
      const res = await fetch(`/api/books/${bookId}`, { method: "DELETE" });
      if (!res.ok) throw new Error("削除に失敗しました");
      toast.success("削除完了", "書籍が正常に削除されました。");
    } catch (err) {
      toast.error("エラー", "書籍の削除に失敗しました。もう一度お試しください。");
    }
  };

  return (
    <button onClick={handleDelete} className="btn-danger">
      削除
    </button>
  );
}
```

## 3.2 ページネーションコンポーネント

```
// components/Pagination.tsx

"use client";

import Link from "next/link";
import { useSearchParams } from "next/navigation";

type PaginationProps = {
  currentPage: number;
  totalPages: number;
  basePath: string; // 例: "/books"
};

export function Pagination({
  currentPage,
  totalPages,
  basePath,
}: PaginationProps) {
  const searchParams = useSearchParams();

  // ページ番号から URL を生成
  const createPageUrl = (page: number): string => {
    const params = new URLSearchParams(searchParams.toString());
    params.set("page", String(page));
    return `${basePath}?${params.toString}`;
  };

  // 表示するページ番号のリストを計算
  // 例: currentPage=5, totalPages=10 → [1, "...", 4, 5, 6, "...", 10]
  const getPageNumbers = (): (number | "ellipsis")[] => {
    const pages: (number | "ellipsis")[] = [];
    const maxVisible = 5; // 表示するページ番号の最大数（省略記号除く）

    if (totalPages <= maxVisible + 2) {
      // 全ページ数が少ない場合、すべて表示
      for (let i = 1; i <= totalPages; i++) {
        pages.push(i);
      }
    } else {
      // 常に最初のページを表示
      pages.push(1);

      // 現在のページが最初の方にある場合
      if (currentPage <= 3) {
        pages.push(2, 3, 4);
        pages.push("ellipsis");
      }

      // 現在のページが最後の方にある場合
    }
  };
}
```

```

    else if (currentPage >= totalPages - 2) {
      pages.push("ellipsis");
      pages.push(totalPages - 3, totalPages - 2, totalPages - 1);
    }
    // 現在のページが中間にある場合
    else {
      pages.push("ellipsis");
      pages.push(currentPage - 1, currentPage, currentPage + 1);
      pages.push("ellipsis");
    }

    // 常に最後のページを表示
    pages.push(totalPages);
  }

  return pages;
};

// ページが1ページしかない場合は表示しない
if (totalPages <= 1) return null;

const pageNumbers = getPageNumbers();
const isFirstPage = currentPage === 1;
const isLastPage = currentPage === totalPages;

// 共通のページ番号ボタンスタイル
const basePageClass = `
  inline-flex h-10 w-10 items-center justify-center
  rounded-lg text-sm font-medium
  transition-all duration-200
  focus-visible:outline-none focus-visible:ring-2
  focus-visible:ring-primary-500 focus-visible:ring-offset-2
`;

const activePageClass = `
  ${basePageClass}
  bg-primary-600 text-white shadow-md
  dark:bg-primary-500
`;

const inactivePageClass = `
  ${basePageClass}
  text-gray-600 hover:bg-gray-100
  dark:text-gray-400 dark:hover:bg-gray-800
`;

const disabledNavClass = `
  inline-flex h-10 items-center justify-center
  rounded-lg px-3 text-sm font-medium
  text-gray-300 cursor-not-allowed

```



```

    dark:text-gray-600
  `;

const enabledNavClass = `
  inline-flex h-10 items-center justify-center
  rounded-lg px-3 text-sm font-medium
  text-gray-600 hover:bg-gray-100
  transition-all duration-200
  dark:text-gray-400 dark:hover:bg-gray-800
  focus-visible:outline-none focus-visible:ring-2
  focus-visible:ring-primary-500 focus-visible:ring-offset-2
`;

return (
  <nav
    role="navigation"
    aria-label="ページネーション"
    className="flex items-center justify-center gap-1 py-8"
  >
    { /* 前のページボタン */ }
    {isFirstPage ? (
      <span className={disabledNavClass} aria-disabled="true">
        <svg
          className="mr-1 h-4 w-4"
          fill="none"
          viewBox="0 0 24 24"
          stroke="currentColor"
          strokeWidth={2}
          aria-hidden="true"
        >
          <path
            strokeLinecap="round"
            strokeLinejoin="round"
            d="M15.75 19.5L8.25 12l7.5-7.5"
          />
        </svg>
        前へ
      </span>
    ) : (
      <Link
        href={createPageUrl(currentPage - 1)}
        className={enabledNavClass}
        aria-label="前のページへ"
      >
        <svg
          className="mr-1 h-4 w-4"
          fill="none"
          viewBox="0 0 24 24"
          stroke="currentColor"
          strokeWidth={2}

```

```

        aria-hidden="true"
      >
      <path
        strokeLinecap="round"
        strokeLinejoin="round"
        d="M15.75 19.5L8.25 12L7.5-7.5"
      />
    </svg>
    前へ
  </Link>
})}

{/* ページ番号リスト */}
<div className="flex items-center gap-1">
  {pageNumbers.map((page, index) => {
    if (page === "ellipsis") {
      return (
        <span
          key={`ellipsis-${index}`}
          className="inline-flex h-10 w-10 items-center justify-center text-gray-400"
          aria-hidden="true"
        >
          ...
        </span>
      );
    }
  })}

  const isActive = page === currentPage;

  return isActive ? (
    <span
      key={page}
      className={activePageClass}
      aria-current="page"
      aria-label={` ${page}ページ目 (現在のページ) `}
    >
      {page}
    </span>
  ) : (
    <Link
      key={page}
      href={createPageUrl(page)}
      className={inactivePageClass}
      aria-label={` ${page}ページ目へ`}
    >
      {page}
    </Link>
  );
})}
</div>

```

```

    { /* 次のページボタン */ }
    {isLastPage ? (
      <span className={disabledNavClass} aria-disabled="true">
        次へ
        <svg
          className="ml-1 h-4 w-4"
          fill="none"
          viewBox="0 0 24 24"
          stroke="currentColor"
          strokeWidth={2}
          aria-hidden="true"
        >
          <path
            strokeLinecap="round"
            strokeLinejoin="round"
            d="M8.25 4.5l7.5 7.5-7.5 7.5"
          />
        </svg>
      </span>
    ) : (
      <Link
        href={createPageUrl(currentPage + 1)}
        className={enabledNavClass}
        aria-label="次のページへ"
      >
        次へ
        <svg
          className="ml-1 h-4 w-4"
          fill="none"
          viewBox="0 0 24 24"
          stroke="currentColor"
          strokeWidth={2}
          aria-hidden="true"
        >
          <path
            strokeLinecap="round"
            strokeLinejoin="round"
            d="M8.25 4.5l7.5 7.5-7.5 7.5"
          />
        </svg>
      </Link>
    )}
  </nav>
);
}

```

### 3.3 空状態（データがない場合）の改善

```
// components/EmptyState.tsx

"use client";

import Link from "next/link";

type EmptyStateProps = {
  title?: string;
  message?: string;
  actionLabel?: string;
  actionHref?: string;
  icon?: "book" | "search" | "error";
};

export function EmptyState({
  title = "書籍が見つかりません",
  message = "まだ書籍が登録されていません。最初の一冊を登録してみましょう。",
  actionLabel = "書籍を登録する",
  actionHref = "/books/new",
  icon = "book",
}: EmptyStateProps) {
  // アイコンの描画
  const renderIcon = () => {
    switch (icon) {
      case "book":
        return (
          <svg
            className="h-16 w-16"
            fill="none"
            viewBox="0 0 24 24"
            stroke="currentColor"
            strokeWidth={1}
            aria-hidden="true"
          >
            <path
              strokeLinecap="round"
              strokeLinejoin="round"
              d="M12 6.042A8.967 8.967 0 006 3.75c-1.052 0-2.062.18-3 .512v14.25A8.987 8.987 0 0012 18.518a8.967 8.967 0 003.958-1.518A8.987 8.987 0 0012 16.518z"
            />
          </svg>
        );
      case "search":
        return (
          <svg
            className="h-16 w-16"
            fill="none"
            viewBox="0 0 24 24"
            stroke="currentColor"
            strokeWidth={1}
            aria-hidden="true"
          >
            <path
              strokeLinecap="round"
              strokeLinejoin="round"
              d="M21 21l-6 6m7-7a7 7 0 10-14 0 7 7 0 0014 0z"
            />
          </svg>
        );
    }
  };

  return (
    <div>
      <h2>{title}</h2>
      <p>{message}</p>
      <p><a href={actionHref}>{actionLabel}</a></p>
      <img alt={renderIcon()} />
    </div>
  );
}
```

```

        strokeWidth={1}
        aria-hidden="true"
      >
        <path
          strokeLinecap="round"
          strokeLinejoin="round"
          d="M21 21l-5.197-5.197m0 0A7.5 7.5 0 105.196 5.196a7.5 7.5 0 0010.607 10.607"
        />
      </svg>
    );
  case "error":
    return (
      <svg
        className="h-16 w-16"
        fill="none"
        viewBox="0 0 24 24"
        stroke="currentColor"
        strokeWidth={1}
        aria-hidden="true"
      >
        <path
          strokeLinecap="round"
          strokeLinejoin="round"
          d="M12 9v3.75m9-.75a9 9 0 11-18 0 9 9 0 0118 0zm-9 3.75h.008v.008H12v-.008z"
        />
      </svg>
    );
  }
};

return (
  <div
    className="
      animate-fade-in
      flex flex-col items-center justify-center
      rounded-xl
      border-2 border-dashed border-gray-200
      bg-gray-50/50
      px-6 py-16
      text-center
      dark:border-gray-700
      dark:bg-gray-800/50
    "
  >
    { /* アイコン */ }
    <div
      className="
        mb-4
        rounded-full
        bg-gray-100

```

```

        p-4
        text-gray-400
        dark:bg-gray-700
        dark:text-gray-500
    "
>
    {renderIcon()}
</div>

{ /* タイトル */ }
<h3
    className="
        mb-2
        text-lg font-semibold
        text-gray-900
        dark:text-gray-100
    "
>
    {title}
</h3>

{ /* メッセージ */ }
<p
    className="
        mb-6
        max-w-sm
        text-sm text-gray-500
        dark:text-gray-400
    "
>
    {message}
</p>

{ /* アクションボタン */ }
{actionHref && (
    <Link href={actionHref} className="btn-primary gap-2">
        <svg
            className="h-4 w-4"
            fill="none"
            viewBox="0 0 24 24"
            stroke="currentColor"
            strokeWidth={2}
            aria-hidden="true"
        >
            <path
                strokeLinecap="round"
                strokeLinejoin="round"
                d="M12 4.5v15m7.5-7.5h-15"
            />
        </svg>

```

```
        {actionLabel}  
      </Link>  
    })  
  </div>  
);  
}
```

### 3.4 ローディングスケルトン

```
// components/BookCardSkeleton.tsx

export function BookCardSkeleton() {
  return (
    <div className="card overflow-hidden">
      {/* サムネイルスケルトン */}
      <div
        className="
          aspect-[3/4] w-full
          bg-gradient-to-r from-gray-200 via-gray-100 to-gray-200
          bg-[length:200%_100%]
          animate-shimmer
          dark:from-gray-700 dark:via-gray-600 dark:to-gray-700
        "
      />

      {/* テキストスケルトン */}
      <div className="flex flex-col gap-2 p-4">
        {/* タイトル行 */}
        <div
          className="
            h-4 w-3/4 rounded
            bg-gradient-to-r from-gray-200 via-gray-100 to-gray-200
            bg-[length:200%_100%]
            animate-shimmer
            dark:from-gray-700 dark:via-gray-600 dark:to-gray-700
          "
        />
        {/* 著者行 */}
        <div
          className="
            h-3 w-1/2 rounded
            bg-gradient-to-r from-gray-200 via-gray-100 to-gray-200
            bg-[length:200%_100%]
            animate-shimmer
            dark:from-gray-700 dark:via-gray-600 dark:to-gray-700
          "
          style={{ animationDelay: "0.1s" }}
        />
        {/* 評価行 */}
        <div
          className="
            mt-2 h-3 w-1/3 rounded
            bg-gradient-to-r from-gray-200 via-gray-100 to-gray-200
            bg-[length:200%_100%]
            animate-shimmer
            dark:from-gray-700 dark:via-gray-600 dark:to-gray-700
          "
        />
      </div>
    </div>
  )
}
```



```

        style={{ animationDelay: "0.2s" }}
      />
    </div>
  </div>
);
}

export function BookGridSkeleton({ count = 8 }: { count?: number }) {
  return (
    <div
      className="
        grid grid-cols-1 gap-4
        sm:grid-cols-2 sm:gap-5
        lg:grid-cols-3 lg:gap-6
        xl:grid-cols-4
      "
    >
      {Array.from({ length: count }).map((_, i) => (
        <BookCardSkeleton key={i} />
      ))}
    </div>
  );
}

```

## 4. アニメーション

### 4.1 Tailwind のトランジションクラス

Tailwind CSS にはトランジション用のユーティリティクラスが組み込まれています。

| クラス                               | 説明                  | CSS 出力                                                                       |
|-----------------------------------|---------------------|------------------------------------------------------------------------------|
| <code>transition</code>           | 一般的なプロパティにトランジション適用 | <code>transition-property: color, background-color, border-color, ...</code> |
| <code>transition-all</code>       | 全プロパティにトランジション適用    | <code>transition-property: all</code>                                        |
| <code>transition-colors</code>    | 色関連のみ               | <code>transition-property: color, background-color, ...</code>               |
| <code>transition-opacity</code>   | 透明度のみ               | <code>transition-property: opacity</code>                                    |
| <code>transition-transform</code> | 変形のみ                | <code>transition-property: transform</code>                                  |
| <code>duration-150</code>         | 150ms               | <code>transition-duration: 150ms</code>                                      |
| <code>duration-200</code>         | 200ms               | <code>transition-duration: 200ms</code>                                      |
| <code>duration-300</code>         | 300ms               | <code>transition-duration: 300ms</code>                                      |
| <code>duration-500</code>         | 500ms               | <code>transition-duration: 500ms</code>                                      |
| <code>ease-in</code>              | 加速カーブ               | <code>transition-timing-function: cubic-bezier(0.4, 0, 1, 1)</code>          |
| <code>ease-out</code>             | 減速カーブ               | <code>transition-timing-function: cubic-bezier(0, 0, 0.2, 1)</code>          |
| <code>ease-in-out</code>          | 加速→減速カーブ            | <code>transition-timing-function: cubic-bezier(0.4, 0, 0.2, 1)</code>        |

## 4.2 カードホバーエフェクト

```
// components/BookCard.tsx 内のホバーエフェクト部分

// カード全体のホバー（上に浮かぶ + 影が深くなる）
<article
  className="
    card
    overflow-hidden
    transition-all duration-300 ease-out
    hover:-translate-y-1.5
    hover:shadow-card-hover
  "
>
  {/* サムネイル画像のホバー（ズームイン） */}
  <div className="relative aspect-[3/4] overflow-hidden">
    <img
      src={book.thumbnailUrl}
      alt={book.title}
      className="
        h-full w-full object-cover
        transition-transform duration-500 ease-out
        group-hover:scale-110
      "
    />

    {/* オーバーレイ（ホバー時にフェードイン） */}
    <div
      className="
        absolute inset-0
        bg-gradient-to-t from-black/60 via-transparent to-transparent
        opacity-0
        transition-opacity duration-300
        group-hover:opacity-100
      "
    />

    {/* 「詳細を見る」テキスト（ホバー時に下からスライドイン） */}
    <div
      className="
        absolute bottom-0 left-0 right-0
        p-4
        text-white
        translate-y-4 opacity-0
        transition-all duration-300
        group-hover:translate-y-0 group-hover:opacity-100
      "
    >
      <span className="text-sm font-medium">詳細を見る →</span>
    </div>
  </div>
```

```
</div>  
</article>
```

## 4.3 ページ遷移時のアニメーション

Next.js App Router でのページ遷移時にフェードイン・スライドインのアニメーションを付けます。

```
// components/PageTransition.tsx

"use client";

import { usePathname } from "next/navigation";
import { useEffect, useState, type ReactNode } from "react";

type PageTransitionProps = {
  children: ReactNode;
};

export function PageTransition({ children }: PageTransitionProps) {
  const pathname = usePathname();
  const [isVisible, setIsVisible] = useState(false);
  const [displayChildren, setDisplayChildren] = useState(children);

  useEffect(() => {
    // パスが変わるたびにアニメーションをリセット
    setIsVisible(false);
    setDisplayChildren(children);

    // 少し遅延させてからフェードイン
    const timer = requestAnimationFrame(() => {
      setIsVisible(true);
    });

    return () => cancelAnimationFrame(timer);
  }, [pathname, children]);

  return (
    <div
      className={`
        transition-all duration-300 ease-out
        ${
          isVisible
            ? "translate-y-0 opacity-100"
            : "translate-y-2 opacity-0"
        }
      `}
    >
      {displayChildren}
    </div>
  );
}
```

// app/layout.tsx での使用例

```
import { PageTransition } from "@components/PageTransition";
import { ToastProvider } from "@contexts/ToastContext";
import { ToastContainer } from "@components/Toast";
import { Header } from "@components/Header";
import { Footer } from "@components/Footer";
import "./globals.css";

export default function RootLayout({
  children,
}: {
  children: React.ReactNode;
}) {
  return (
    <html lang="ja" suppressHydrationWarning>
      <body className="flex min-h-screen flex-col">
        <ToastProvider>
          <Header />
          <main className="container flex-1 py-8">
            <PageTransition>{children}</PageTransition>
          </main>
          <Footer />
          <ToastContainer />
        </ToastProvider>
      </body>
    </html>
  );
}
```

## 4.4 ボタンのアニメーション

```
// components/AnimatedButton.tsx

"use client";

import { type ButtonHTMLAttributes, type ReactNode } from "react";

type AnimatedButtonProps = ButtonHTMLAttributes<HTMLButtonElement> & {
  variant?: "primary" | "secondary" | "danger" | "ghost";
  size?: "sm" | "md" | "lg";
  isLoading?: boolean;
  children: ReactNode;
};

export function AnimatedButton({
  variant = "primary",
  size = "md",
  isLoading = false,
  children,
  className = "",
  disabled,
  ...props
}: AnimatedButtonProps) {
  // バリエーションごとのスタイル
  const variantStyles = {
    primary: `
      bg-primary-600 text-white
      hover:bg-primary-700
      active:bg-primary-800
      focus-visible:ring-primary-500
      shadow-md hover:shadow-lg
    `,
    secondary: `
      bg-secondary-600 text-white
      hover:bg-secondary-700
      active:bg-secondary-800
      focus-visible:ring-secondary-500
    `,
    danger: `
      bg-red-600 text-white
      hover:bg-red-700
      active:bg-red-800
      focus-visible:ring-red-500
    `,
    ghost: `
      text-gray-600 hover:bg-gray-100
      dark:text-gray-400 dark:hover:bg-gray-800
      focus-visible:ring-gray-500
    `
  };
}
```

```

};

// サイズごとのスタイル
const sizeStyles = {
  sm: "px-3 py-1.5 text-xs gap-1.5",
  md: "px-4 py-2.5 text-sm gap-2",
  lg: "px-6 py-3 text-base gap-2.5",
};

return (
  <button
    className={`
      inline-flex items-center justify-center
      rounded-lg font-medium
      transition-all duration-200 ease-out
      focus-visible:outline-none focus-visible:ring-2 focus-visible:ring-offset-2
      disabled:pointer-events-none disabled:opacity-50
      active:scale-[0.97]
      ${variantStyles[variant]}
      ${sizeStyles[size]}
      ${className}
    `}
    disabled={disabled || isLoading}
    {...props}
  >
    {isLoading && (
      <svg
        className="h-4 w-4 animate-spin"
        viewBox="0 0 24 24"
        fill="none"
        aria-hidden="true"
      >
        <circle
          className="opacity-25"
          cx="12"
          cy="12"
          r="10"
          stroke="currentColor"
          strokeWidth="4"
        />
        <path
          className="opacity-75"
          fill="currentColor"
          d="M4 12a8 8 0 018-8V0C5.373 0 0 5.373 0 12h4z"
        />
      </svg>
    )}
    {children}
  </button>

```



```
);  
}
```

## 5. ダークモード（発展）

### 5.1 Tailwind のダークモード設定

`tailwind.config.ts` で `darkMode: "class"` を設定済みなので、`<html>` タグに `dark` クラスを付与することでダークモードが有効になります。

```
<!-- ライトモード -->  
<html lang="ja">  
  
<!-- ダークモード -->  
<html lang="ja" class="dark">
```

### 5.2 dark: クラスの使い方

Tailwind の `dark:` バリエントを使うと、ダークモード時に適用されるスタイルを簡単に記述できます。

```
{/* 基本パターン: 通常のクラス後に dark: プレフィックスを付ける */}  
<div className="bg-white dark:bg-gray-900">  
  <h1 className="text-gray-900 dark:text-gray-100">タイトル</h1>  
  <p className="text-gray-600 dark:text-gray-400">本文テキスト</p>  
</div>  
  
{/* ボーダーの例 */}  
<div className="border border-gray-200 dark:border-gray-700">  
  ...  
</div>  
  
{/* ホバーとの組み合わせ */}  
<button className="  
  bg-blue-500 hover:bg-blue-600  
  dark:bg-blue-600 dark:hover:bg-blue-700  
">  
  ボタン  
</button>  
  
{/* リングとフォーカスとの組み合わせ */}  
<input className="  
  focus:ring-blue-500  
  dark:focus:ring-blue-400  
" />
```

## 5.3 テーマ切り替えボタンの実装

```
// components/ThemeToggle.tsx

"use client";

import { useState, useEffect } from "react";

type Theme = "light" | "dark" | "system";

export function ThemeToggle() {
  const [theme, setTheme] = useState<Theme>("system");
  const [mounted, setMounted] = useState(false);

  // コンポーネントがマウントされた後にテーマを読み込む
  // (SSR 時にミスマッチが起きるのを防ぐため)
  useEffect(() => {
    setMounted(true);

    // localStorage から保存済みのテーマを読み込む
    const savedTheme = localStorage.getItem("theme") as Theme | null;
    if (savedTheme) {
      setTheme(savedTheme);
      applyTheme(savedTheme);
    } else {
      applyTheme("system");
    }
  }, []);

  // システムのカラースキーム変更を監視
  useEffect(() => {
    if (theme !== "system") return;

    const mediaQuery = window.matchMedia("(prefers-color-scheme: dark)");
    const handleChange = () => applyTheme("system");

    mediaQuery.addEventListener("change", handleChange);
    return () => mediaQuery.removeEventListener("change", handleChange);
  }, [theme]);

  // テーマを実際に適用する関数
  const applyTheme = (newTheme: Theme) => {
    const root = document.documentElement;
    const isDark =
      newTheme === "dark" ||
      (newTheme === "system" &&
        window.matchMedia("(prefers-color-scheme: dark)").matches);

    if (isDark) {
      root.classList.add("dark");
    }
  };
}
```

```

    } else {
      root.classList.remove("dark");
    }
  };

  // テーマを切り替える関数
  const toggleTheme = () => {
    const nextTheme: Theme =
      theme === "light" ? "dark" : theme === "dark" ? "system" : "light";

    setTheme(nextTheme);
    localStorage.setItem("theme", nextTheme);
    applyTheme(nextTheme);
  };

  // マウント前はプレースホルダーを表示（ハイドレーションエラー防止）
  if (!mounted) {
    return (
      <button
        className="
          inline-flex h-10 w-10 items-center justify-center
          rounded-lg
          text-gray-500
        "
        aria-label="テーマ切り替え"
      >
        <div className="h-5 w-5" />
      </button>
    );
  }

  // テーマに応じたアイコン
  const themeIcon = () => {
    switch (theme) {
      case "light":
        return (
          <svg
            className="h-5 w-5"
            fill="none"
            viewBox="0 0 24 24"
            stroke="currentColor"
            strokeWidth={2}
            aria-hidden="true"
          >
            <path
              strokeLinecap="round"
              strokeLinejoin="round"
              d="M12 3v2.25m6.364.386l-1.591 1.591M21 12h-2.25m-.386 6.364l-1.591-1.591M12 21v-2.25m-6.364-.386l1.591-1.591"
            />
          </svg>
        );
      case "dark":
        return (
          <svg
            className="h-5 w-5"
            fill="none"
            viewBox="0 0 24 24"
            stroke="currentColor"
            strokeWidth={2}
            aria-hidden="true"
          >
            <path
              strokeLinecap="round"
              strokeLinejoin="round"
              d="M21 12.75a9 9 0 11-18 0 9 9 0 0118 0z"
            />
          </svg>
        );
    }
  };

```

```

    );
    case "dark":
        return (
            <svg
                className="h-5 w-5"
                fill="none"
                viewBox="0 0 24 24"
                stroke="currentColor"
                strokeWidth={2}
                aria-hidden="true"
            >
                <path
                    strokeLinecap="round"
                    strokeLinejoin="round"
                    d="M21.752 15.002A9.718 9.718 0 0118 15.75c-5.385 0-9.75-4.365-9.75-9.75 0-1
                />
            </svg>
        );
    case "system":
        return (
            <svg
                className="h-5 w-5"
                fill="none"
                viewBox="0 0 24 24"
                stroke="currentColor"
                strokeWidth={2}
                aria-hidden="true"
            >
                <path
                    strokeLinecap="round"
                    strokeLinejoin="round"
                    d="M9 17.25v1.007a3 3 0 01-.879 2.122L7.5 21h9l-.621-.621A3 3 0 0115 18.257V
                />
            </svg>
        );
    }
};

// テーマのラベル
const themeLabel =
    theme === "light" ? "ライト" : theme === "dark" ? "ダーク" : "システム";

return (
    <button
        onClick={toggleTheme}
        className="
            inline-flex h-10 items-center justify-center gap-2
            rounded-lg px-3
            text-gray-600
            transition-colors duration-200

```

```

    hover:bg-gray-100 hover:text-gray-900
    dark:text-gray-400
    dark:hover:bg-gray-800 dark:hover:text-gray-100
    focus-visible:outline-none focus-visible:ring-2
    focus-visible:ring-primary-500 focus-visible:ring-offset-2
  "
  aria-label={`テーマ切り替え（現在: ${themeLabel}モード）`}
  title={` ${themeLabel}モード`}
>
  {themeIcon()}
  <span className="hidden text-sm font-medium sm:inline">
    {themeLabel}
  </span>
</button>
);
}

```

### ちらつき防止のスクリプト:

ページ読み込み時にダークモードが一瞬ライトモードで表示されるのを防ぐため、`<head>` 内にインラインスクリプトを設置します。

```

// app/layout.tsx の <head> 内に追加

<script
  dangerouslySetInnerHTML={{
    __html: `
      (function() {
        try {
          var theme = localStorage.getItem('theme');
          var isDark = theme === 'dark' ||
            (!theme && window.matchMedia('(prefers-color-scheme: dark)').matches);
          if (isDark) {
            document.documentElement.classList.add('dark');
          }
        } catch (e) {}
      })();
    `,
  }}
/>

```

## 6. アクセシビリティ

---

### 6.1 aria 属性の追加

アクセシビリティを確保するために、各コンポーネントに適切な ARIA 属性を追加します。

```
// components/Header.tsx

"use client";

import { useState } from "react";
import Link from "next/link";
import { usePathname } from "next/navigation";
import { ThemeToggle } from "../ThemeToggle";

export function Header() {
  const pathname = usePathname();
  const [isMobileMenuOpen, setIsMobileMenuOpen] = useState(false);

  const navItems = [
    { href: "/", label: "ホーム" },
    { href: "/books", label: "書籍一覧" },
    { href: "/books/new", label: "新規登録" },
  ];

  return (
    <header
      className="
        sticky top-0 z-40
        border-b border-[var(--color-border)]
        glass
      "
      role="banner"
    >
      <div className="container flex h-16 items-center justify-between">
        { /* □□ */ }
        <Link
          href="/"
          className="
            flex items-center gap-2
            text-xl font-bold
            text-[var(--color-foreground)]
            transition-colors hover:text-primary-600
            dark:hover:text-primary-400
          "
          aria-label="BookShelf ホームへ"
        >
          <svg
            className="h-7 w-7 text-primary-600 dark:text-primary-400"
            fill="none"
            viewBox="0 0 24 24"
            stroke="currentColor"
            strokeWidth={2}
            aria-hidden="true"
          >

```

```

    <path
      strokeLinecap="round"
      strokeLinejoin="round"
      d="M12 6.042A8.967 8.967 0 006 3.75c-1.052 0-2.062.18-3 .512v14.25A8.987 8.9
    />
  </svg>
  BookShelf
</Link>

{/* デスクトップナビゲーション */}
<nav
  className="hidden items-center gap-1 md:flex"
  role="navigation"
  aria-label="メインナビゲーション"
>
  {navItems.map((item) => {
    const isActive = pathname === item.href;
    return (
      <Link
        key={item.href}
        href={item.href}
        className={`
          rounded-lg px-3 py-2 text-sm font-medium
          transition-colors duration-200
          ${
            isActive
              ? "bg-primary-50 text-primary-700 dark:bg-primary-950 dark:text-prin
              : "text-gray-600 hover:bg-gray-100 hover:text-gray-900 dark:text-gra
          }
        `}
        aria-current={isActive ? "page" : undefined}
      >
        {item.label}
      </Link>
    );
  })}
</nav>

{/* 右側のアクション */}
<div className="flex items-center gap-2">
  <ThemeToggle />

  {/* モバイルメニューボタン */}
  <button
    className="
      inline-flex h-10 w-10 items-center justify-center
      rounded-lg
      text-gray-600
      transition-colors
      hover:bg-gray-100

```



```

        dark:text-gray-400 dark:hover:bg-gray-800
        md:hidden
    "
    onClick={() => setIsMobileMenuOpen(!isMobileMenuOpen)}
    aria-expanded={isMobileMenuOpen}
    aria-controls="mobile-menu"
    aria-label={isMobileMenuOpen ? "メニューを閉じる" : "メニューを開く"}
>
    {isMobileMenuOpen ? (
        <svg
            className="h-6 w-6"
            fill="none"
            viewBox="0 0 24 24"
            stroke="currentColor"
            strokeWidth={2}
            aria-hidden="true"
        >
            <path strokeLinecap="round" strokeLinejoin="round" d="M6 18L18 6M6 6l12 12" />
        </svg>
    ) : (
        <svg
            className="h-6 w-6"
            fill="none"
            viewBox="0 0 24 24"
            stroke="currentColor"
            strokeWidth={2}
            aria-hidden="true"
        >
            <path strokeLinecap="round" strokeLinejoin="round" d="M3.75 6.75h16.5M3.75 12.75h16.5" />
        </svg>
    )}
</button>
</div>
</div>

{/* モバイルメニュー */}
{isMobileMenuOpen && (
    <nav
        id="mobile-menu"
        className="
            border-t border-[var(--color-border)]
            bg-[var(--color-background)]
            p-4
            md:hidden
        "
        role="navigation"
        aria-label="モバイルナビゲーション"
    >
        <ul className="flex flex-col gap-1" role="list">
            {navItems.map((item) => {

```

```

const isActive = pathname === item.href;
return (
  <li key={item.href}>
    <Link
      href={item.href}
      className={`
        block rounded-lg px-4 py-3 text-sm font-medium
        transition-colors duration-200
        ${
          isActive
            ? "bg-primary-50 text-primary-700 dark:bg-primary-950 dark:text-
            : "text-gray-600 hover:bg-gray-50 dark:text-gray-400 dark:ho
        }
      `}
      aria-current={isActive ? "page" : undefined}
      onClick={() => setIsMobileMenuOpen(false)}
    >
      {item.label}
    </Link>
  </li>
);
}}
</ul>
</nav>
)}
</header>
);
}

```

## 6.2 キーボードナビゲーション

キーボードのみでアプリを操作できるようにするためのポイントと実装です。

```

// components/BookCard.tsx に追加するキーボード対応

// カードが Link で囲まれている場合、Enter/Space で遷移が可能 (Link のデフォルト動作)
// それ以外のインタラクティブ要素にも対応する

// 例: 削除確認ダイアログのキーボード対応
// components/ConfirmDialog.tsx

"use client";

import { useEffect, useRef, type ReactNode } from "react";

type ConfirmDialogProps = {
  isOpen: boolean;
  title: string;
  message: string;
  confirmLabel?: string;
  cancelLabel?: string;
  variant?: "danger" | "primary";
  onConfirm: () => void;
  onCancel: () => void;
};

export function ConfirmDialog({
  isOpen,
  title,
  message,
  confirmLabel = "確認",
  cancelLabel = "キャンセル",
  variant = "primary",
  onConfirm,
  onCancel,
}: ConfirmDialogProps) {
  const dialogRef = useRef<HTMLDivElement>(null);
  const cancelButtonRef = useRef<HTMLButtonElement>(null);

  // ダイアログが開いたらキャンセルボタンにフォーカス
  useEffect(() => {
    if (isOpen) {
      cancelButtonRef.current?.focus();
    }
  }, [isOpen]);

  // Escape キーで閉じる
  useEffect(() => {
    const handleKeyDown = (e: KeyboardEvent) => {
      if (!isOpen) return;

      if (e.key === "Escape") {

```

```

    onCancel();
  }

  // Tab キーのフォーカストラップ
  if (e.key === "Tab" && dialogRef.current) {
    const focusableElements = dialogRef.current.querySelectorAll<HTMLElement>(
      'button, [href], input, select, textarea, [tabindex]:not([tabindex="-1"])'
    );
    const firstElement = focusableElements[0];
    const lastElement = focusableElements[focusableElements.length - 1];

    if (e.shiftKey) {
      if (document.activeElement === firstElement) {
        e.preventDefault();
        lastElement.focus();
      }
    } else {
      if (document.activeElement === lastElement) {
        e.preventDefault();
        firstElement.focus();
      }
    }
  }
};

document.addEventListener("keydown", handleKeyDown);
return () => document.removeEventListener("keydown", handleKeyDown);
}, [isOpen, onCancel]);

// body のスクロールを無効化
useEffect(() => {
  if (isOpen) {
    document.body.style.overflow = "hidden";
  } else {
    document.body.style.overflow = "";
  }
  return () => {
    document.body.style.overflow = "";
  };
}, [isOpen]);

if (!isOpen) return null;

return (
  <div
    className="
      fixed inset-0 z-50
      flex items-center justify-center
      p-4
    "

```

```

    role="dialog"
    aria-modal="true"
    aria-labelledby="dialog-title"
    aria-describedby="dialog-description"
  >
    {/* オーバーレイ */}
    <div
      className="
        absolute inset-0
        bg-black/50
        animate-fade-in
        backdrop-blur-sm
        "
      onClick={onCancel}
      aria-hidden="true"
    />

    {/* ダイアログ本体 */}
    <div
      ref={dialogRef}
      className="
        relative
        w-full max-w-md
        animate-scale-in
        rounded-xl
        bg-[var(--color-card)]
        p-6
        shadow-xl
        "
    >
      <h2
        id="dialog-title"
        className="text-lg font-semibold text-[var(--color-foreground)]"
      >
        {title}
      </h2>

      <p
        id="dialog-description"
        className="mt-2 text-sm text-[var(--color-muted)]"
      >
        {message}
      </p>

      <div className="mt-6 flex justify-end gap-3">
        <button
          ref={cancelButtonRef}
          onClick={onCancel}
          className="btn-ghost"
        >

```

```

        {cancelLabel}
      </button>

      <button
        onClick={onConfirm}
        className={variant === "danger" ? "btn-danger" : "btn-primary"}
      >
        {confirmLabel}
      </button>
    </div>
  </div>
</div>
);
}

```

### 6.3 色のコントラスト

WCAG 2.1 のコントラスト比基準を満たすための指針です。

| レベル         | コントラスト比  | 対象                      |
|-------------|----------|-------------------------|
| AA（通常テキスト）  | 4.5:1 以上 | 本文、ラベル、プレースホルダ          |
| AA（大きいテキスト） | 3:1 以上   | 見出し（18px以上、または14px太字以上） |
| AAA（通常テキスト） | 7:1 以上   | 最高レベルの可読性が求められる場面       |

本アプリで採用しているカラーのコントラスト比:

| 組み合わせ              | ライトモード                     | ダークモード                     | 基準 |
|--------------------|----------------------------|----------------------------|----|
| メインテキスト / 背景       | 15.4:1 (#0f172a / #ffffff) | 16.0:1 (#f8fadc / #0f172a) | AA |
| ミュートテキスト / 背景      | 5.5:1 (#64748b / #ffffff)  | 5.7:1 (#94a3b8 / #0f172a)  | AA |
| プライマリボタン文字 / ボタン背景 | 8.6:1 (#ffffff / #4f46e5)  | --                         | AA |
| エラーテキスト / エラー背景    | 4.6:1 (#991b1b / #fef2f2)  | 5.1:1                      | AA |

確認ツール: - Chrome DevTools: Elements パネルの Color Contrast 表示 - [WebAIM Contrast Checker](#) - axe DevTools ブラウザ拡張

## 7. 最終的な画面の説明

### 7.1 トップページ（書籍一覧）



**ヘッダー:** - 画面上部に `sticky` で固定。スクロールしても常に表示される - `glass` クラスによる半透明のすりガラス効果 ( `backdrop-blur-md` ) - 左にロゴ (本のアイコン + "BookShelf")、中央にナビゲーションリンク、右にテーマ切り替えボタン - モバイルではハンバーガーメニューに切り替わる

**メインコンテンツ:** - ページタイトルと書籍数を表示。右端に「新規登録」ボタン - 検索バーとフィルター（ステータス、並び順） - 書籍カードがレスポンシブグリッドで並ぶ。各カードはホバーで浮き上がるアニメーション付き - カードにはサムネイル、タイトル（最大2行で省略）、著者名、星評価を表示

**フッター:** - 著作権表示、外部リンク

## 7.2 新規登録ページ

BookShelf

ホーム書籍一覧新規登録

[← 書籍一覧に戻る](#)

### 新しい書籍を登録

タイトル \*

書籍のタイトルを入力

著者名 \*

著者名を入力

ISBN

978-...

ステータス \*

読みたい▼

評価

★ ★ ★ ★ ★

(クリックで選択)

メモ

キャンセル

+ 登録する

BookShelf © 2026

[GitHub](#) [Twitter](#)



**レイアウト:** - フォームは `max-w-2xl` で中央配置。カードスタイルで背景と分離 - 「← 書籍一覧に戻る」のパンくずリンクをフォーム上部に配置 - 必須フィールドには `*` マーク。バリデーションエラー時はフィールド下に赤文字でメッセージ表示 - 評価は星アイコンをクリックして選択するインタラクティブ UI - 「登録する」ボタンは送信中にローディングスピナーを表示 - 送信成功時にトースト通知が右上に表示される

## 7.3 詳細ページ



**レイアウト:** - 上段は2カラム: 左に表紙画像（`aspect-[3/4]`）、右に書籍情報 - モバイルでは1カラムに切り替わり、画像が上、情報が下に配置される - ステータスバッジが右上に表示される - 「編集」ボタン（`btn-outline`）と「削除」ボタン（`btn-danger`）を横に並べる - 削除ボタン押下時は `ConfirmDialog` が表示される - メモは別のカードセクションとして表示

## 7.4 編集ページ

BookShelf

ホーム書籍一覧新規登録

[← 書籍詳細に戻る](#)

### 書籍情報を編集

タイトル \*

React実践ガイド

著者名 \*

山田太郎

ISBN

978-4-xxx-xxxxx

ステータス \*

読書中 ▼

評価

★ ★ ★ ★ ☆

メモ

この本は非常に参考になった。特に第3章のアーキテクチャの解説が素晴らしい。

キャンセル

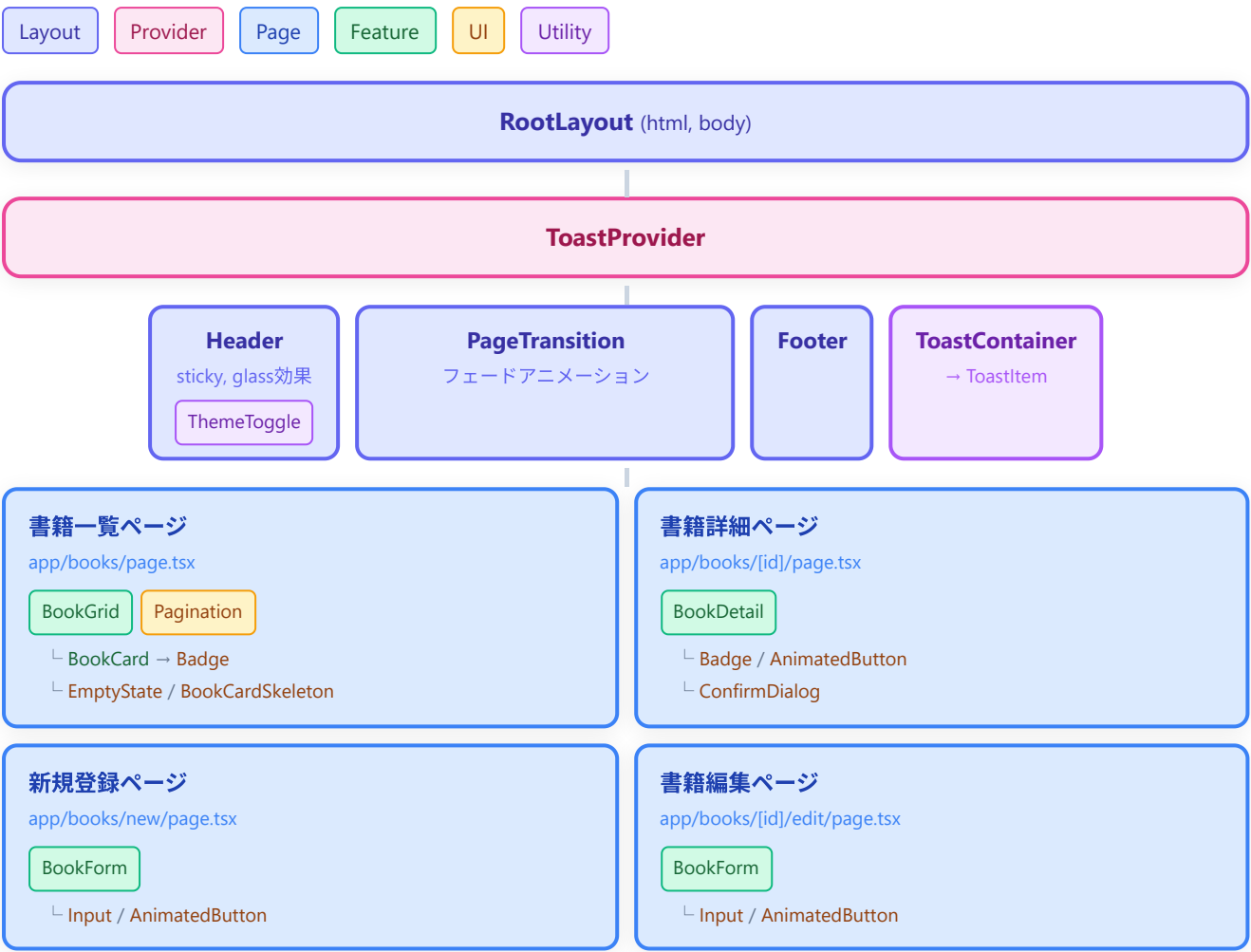
✓ 更新する

BookShelf © 2026

[GitHub](#) [Twitter](#)

**レイアウト:** - 新規登録ページとほぼ同じフォームレイアウトを再利用 - 違いは: タイトルが「書籍情報を編集」、ボタンラベルが「更新する」、パンくずが「← 書籍詳細に戻る」 - フォームの各フィールドには既存データが pre-fill される - 更新成功時にトースト通知が表示され、詳細ページにリダイレクト

# 8. コンポーネント構成図（最終版）



## 構成図の読み方:

| 色     | カテゴリ               | 説明                            |
|-------|--------------------|-------------------------------|
| 青     | Page Components    | Next.js のルートに対応するページコンポーネント   |
| 緑     | Feature Components | 特定の機能を担うコンポーネント（書籍グリッド、フォーム等） |
| 黄     | UI Components      | 再利用可能な汎用 UI 部品                |
| 紫     | Utility Components | トースト通知やテーマ切替などの横断的機能          |
| インディゴ | Layout Components  | ヘッダー、フッター、ページ遷移など画面の骨格        |
| ピンク   | Providers          | React Context による状態管理         |

## まとめ

この章で実装した内容を振り返ります。

| 項目          | 実装内容                              | ファイル                                                                          |
|-------------|-----------------------------------|-------------------------------------------------------------------------------|
| グローバルスタイル   | CSS カスタムプロパティ、ベースレイヤー、コンポーネントレイヤー | <code>app/globals.css</code>                                                  |
| Tailwind 設定 | カスタムカラー、アニメーション、フォント、シャドウ         | <code>tailwind.config.ts</code>                                               |
| レスポンシブグリッド  | 1列→2列→3列→4列のブレイクポイント対応            | <code>components/BookGrid.tsx</code>                                          |
| トースト通知      | Context API ベースの成功/エラー通知          | <code>contexts/ToastContext.tsx</code> ,<br><code>components/Toast.tsx</code> |
| ページネーション    | ページ番号計算ロジック付きナビゲーション              | <code>components/Pagination.tsx</code>                                        |
| 空状態         | アイコン + メッセージ + CTA ボタン            | <code>components/EmptyState.tsx</code>                                        |
| スケルトン       | シマアニメーション付きローディング                 | <code>components/BookCardSkeleton.tsx</code>                                  |
| カードアニメーション  | ホバーで浮き上がり + 画像ズーム + オーバーレイ        | <code>components/BookCard.tsx</code>                                          |
| ページ遷移       | フェードイン・スライドアップ                    | <code>components/PageTransition.tsx</code>                                    |
| ダークモード      | class ベース切替 + ちらつき防止スクリプト         | <code>components/ThemeToggle.tsx</code>                                       |
| アクセシビリティ    | aria属性、キーボードナビ、フォーカストラップ、コントラスト比  | 各コンポーネント                                                                      |
| 確認ダイアログ     | モーダル + フォーカストラップ + Escape 対応      | <code>components/ConfirmDialog.tsx</code>                                     |

次の章では、テストの書き方とデプロイについて学びます。